ME5418 - Neural Network C&R

Team 41: Benjamin Teh Jhen Hing, Lu Jia Yih Jensen, Wee Fook Choon

1 Code Overview

1.1 Overview, Existing Libraries & Design Motivation

In this project, a Deep Q-Network (`DQN`) is implemented in PyTorch (`./src/neural_nets/dueling_ddqn_torch.py`) to approximate the action-value function $Q(s, a)$, estimating the expected discounted return of executing action a in state s. While our proposal initially considered Proximal Policy Optimization (PPO) for its native `MultiDiscrete([9, 2, 2])` support, we adopted **Dueling Double DQN (Dueling DDQN)** as our primary architecture because the action space flattens losslessly into $9 \times 2 \times 2 = 36$ discrete actions (`Discrete(36)`), and off-policy experience replay is more sample-efficient when each 60 Hz agent step integrates four 240 Hz physics sub-steps.

Existing Libraries & Code Organization: We build on **PyTorch** (`torch.nn`, `LayerNorm`, `Adam`, `SmoothL1Loss`, autograd) following standard Dueling DDQN patterns, interfaced with **Gymnasium** (`gym.make`) and our custom **Pymunk/Pygame/NumPy** simulator from Assignment 1. The module supports Vanilla DQN (`--dqn`), Double DQN (`--double_dqn`), and Dueling DDQN (`--dueling_ddqn`, default) via two main components:

- `DQN` class: Defines the network architecture (Figure 1) and forward pass, supporting single-stream (`enable_dueling_dqn=False`, 89,636 parameters) and dueling-stream (`enable_dueling_dqn=True`, 150,949 parameters) heads.
- `run_dqn_network()` function: Initializes the headless environment and policy/target networks, runs a forward pass for action selection, computes the Double DQN target, and performs one backward update.

1.2 Dueling DDQN Architecture (`dueling_ddqn_torch.DQN`)

The network maps the 52-dimensional normalized state $s \in [-1, 1]^{52}$ through two fully-connected layers of 256 units (`fc1`, `fc2`), each followed by `LayerNorm` and `ReLU`. Instead of projecting directly to 36 Q-values, the Dueling head splits the 256-unit embedding into two parallel 128-unit streams: (1) a **State-Value stream** $V(s) \in \mathbb{R}^1$ estimating state value, and (2) an **Action-Advantage stream** $A(s, a) \in \mathbb{R}^{36}$ estimating relative action advantages. These are combined via mean-subtracted aggregation $Q(s, a) = V(s) + (A(s, a) - \frac{1}{36} \sum_{a'} A(s, a'))$ for identifiability, totaling **150,949 trainable parameters**.

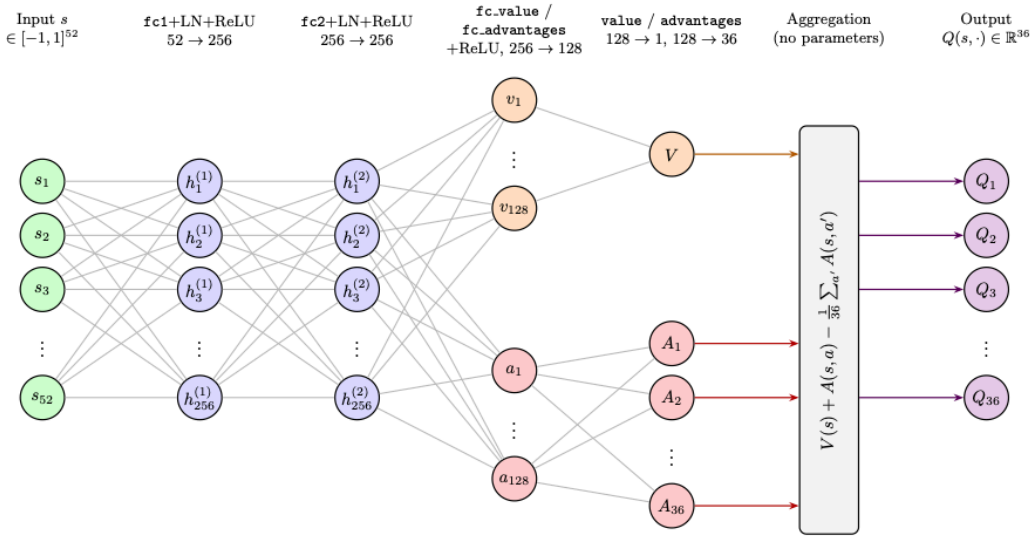


Figure 1: Policy/target network `DQN(state_dim=52, action_dim=36, hidden_dim=256, enable_dueling_dqn=True)`. Green: input; blue: shared backbone; orange: value stream; red: advantage stream; purple: output. With `enable_dueling_dqn=False`, the streams are replaced by a single linear head $Q(s, \cdot) = W_3 h^{(2)} + b_3$, $W_3 \in \mathbb{R}^{36 \times 256}$.

1.3 Forward/Backward Pass Verification (`run_dqn_network`) & Output Analysis

Running `python -m src --demo --dueling_ddqn` from `ME5418_Group41_RocketLeague/` executes Algorithm 1 and outputs three structured verification blocks (values vary per run due to random initialization and spawn):

1. **Network Architecture Summary:** Prints the compute device (`cuda`/`cpu`), parameter count (150,949 trainable parameters), and PyTorch layer hierarchy (`fc1`, `ln1`, `fc2`, `ln2`, `fc_value`, `value`, `fc_advantages`, `advantages`).

Algorithm 1: Single-Step Double DQN Forward & Backward Pass Verification

Data: State dim 52, action dim 36, hidden dim 256, discount factor $\gamma = 0.99$
Result: Single-step Double DQN forward evaluation, TD target calculation, and gradient update
`// 1. Initialisation & 2. Forward pass`
$Q_{\text{policy}} \leftarrow \text{DQN}(52, 36, 256, \text{enable_dueling_dqn} = \text{True})$; $Q_{\text{target}} \leftarrow \text{copy}(Q_{\text{policy}})$;
optimizer $\leftarrow \text{Adam}(Q_{\text{policy}}, \text{lr} = 10^{-3})$; loss_fn $\leftarrow$ SmoothL1Loss (Huber);
$s \leftarrow \text{env.reset}()$; $q \leftarrow Q_{\text{policy}}(s)$; $a \leftarrow \arg\max_{a'} q[a']$; $s', r, \text{done} \leftarrow \text{env.step}(a)$;
`// 3. Backpropagation (Double DQN target)`
$a_{\text{best}} \leftarrow \arg\max_{a'} Q_{\text{policy}}(s')[a']$; $y \leftarrow r + \gamma \cdot (1 - \text{done}) \cdot Q_{\text{target}}(s')[a_{\text{best}}]$;
loss $\leftarrow$ loss_fn$(q[a], y)$; loss.backward(); optimizer.step();

2. **Forward Pass (`Input -> Output`):** Converts the reset observation into a batch tensor of shape `torch.Size([1, 52])`, evaluates $Q_{\text{policy}}(s)$ to produce an output tensor of shape `torch.Size([1, 36])`, and selects the greedy action $a = \arg\max_{a'} Q(s, a')$, decoding it into vehicle controls `(dir, jump, boost)` (e.g., `Action 1 -> dir=(0, 0), jump=False, boost=True`).
3. **Backward Pass (`Loss & Gradients`):** Steps the environment once with action a to observe reward r and next state s', computes the Double DQN target y and Smooth L1 (Huber) loss, and calls `loss.backward()`. It prints the L_2 gradient norm ($\|\nabla_\theta \mathcal{L}\|_2$) for all 16 parameter tensors—confirming finite, non-zero gradients across both streams and the shared backbone—and compares $Q_{\text{policy}}(s, a)$ before and after `optimizer.step()` (e.g., given pre-update $Q = 0.5226$ and target $y = 0.2996$, one step shifts Q downward to -0.6949).

2 Learning Points and Reflection

- **Changes from Proposal & Action Space Simplification:** Moving from PPO to Dueling DDQN required flattening `MultiDiscrete([9, 2, 2])` into `Discrete(36)`. While this simplification enables off-policy replay, it loses the factored policy structure (where jump and boost heads share gradients across directions), meaning our network must learn jump/boost effects across all 36 outputs via the shared backbone.
- **Double & Dueling Mechanisms, Huber Loss & Step-Size Sensitivity:** In vanilla DQN, $\max_{a'} Q_{\text{target}}(s', a')$ introduces positive overestimation bias. Double DQN (https://arxiv.org/pdf/1509.06461) decouples action selection (Q_{policy}) from evaluation (Q_{target}), while Dueling streams (https://arxiv.org/pdf/1511.06581) generalize state value $V(s)$ across states where actions have similar immediate impact (e.g., mid-air coasting). Smooth L1 (Huber) loss caps the per-sample gradient magnitude at 1 when |TD error| > 1. Moreover, our single-step demo revealed that an unbatched Adam step at lr $= 10^{-3}$ overshoots y because all 150k weights shift simultaneously by $\approx \pm$lr, motivating mini-batch updates (batch size $= 64$) and a lower learning rate ($\sim 10^{-4}$) in Assignment 3.
- **Hidden Layer Sizing, `LayerNorm` & State Representation:** Choosing layer widths is largely empirical (using $256 \to 256 \to 128$ as a starting heuristic to be tuned during training). Adding `LayerNorm` normalizes hidden activations per sample independent of batch size—working seamlessly for both batch-size-1 action selection and mini-batch replay—while stabilizing activations against non-stationary TD targets. Finally, earlier environment pre-testing guided our 52-dim input design: pruning redundant opponent wall raycasts reduced `fc1` parameters, while retaining 4 explicit relational vectors (car-to-ball, opponent-to-ball, ball-to-goals) provided essential geometric inductive biases.

3 Next Steps

- **Learning Agent & Curriculum Training:** For Assignment 3, we will integrate the experience replay buffer, ϵ-greedy exploration schedule, and periodic target synchronization to train the policy via curriculum learning: progressing from a stationary opponent to a ball-chaser, our HZISM finite-state machine bot, and self-play.
- **Head-to-Head Comparison with PPO:** Our multi-agent environment also enables head-to-head 1v1 matches to benchmark our off-policy Dueling DDQN against an on-policy Actor-Critic (PPO) agent.

AI Tool Declaration: We used Google Gemini and Claude Code to write and debug code, format paragraphs, and condense the LaTeX layout. We are responsible for the content and quality of the submitted work.